

PROJECT SOUNDING LINE

The Software Verification and Parallel Test Infrastructure System

PART I

Software Verification Architecture

Version 1.1 Amendment Edition

Amendment purpose

Formalize semantic evidence invalidation, carry-forward evidence, failure-classification repair boundaries, environment-qualified evidence, and closure-confidence composition so a narrow repair or unrelated mainline advance does not force ritual re-execution of proof that remains valid.

Effective August 10, 2026 | Document ID CS-SL-P1-001 v1.1

A. Document Control and Amendment Authority

Field	Value
Program	Project Sounding Line
Subsystem	The Software Verification and Parallel Test Infrastructure System
Document	Governing Architecture and Product Requirements Document - Part I, Version 1.1 Amendment Edition
Base document	Version 1.0, July 28, 2026
Version	1.1
Effective date	August 10, 2026
Document ID	CS-SL-P1-001 v1.1
Status	Governing amendment; additive and superseding only where clauses are explicitly named
Amendment trigger	Project Helm Phase 1 closure and repeated cross-project validation/integration churn exposed gaps between intended evidence reuse and operational behavior.

Normative incorporation rule

The Version 1.0 body remains in force in full except where this Version 1.1 amendment explicitly adds, clarifies, or supersedes a clause. This amendment must never be read as permission to weaken required proof. Its purpose is to preserve correctness while preventing redundant proof, avoidable serialization, and closure loops that do not increase confidence.

This amendment is deliberately narrow. It does not change the three-document authority split: Part I defines verification meaning, Part II defines execution machinery, and Part III defines repository/Codex/release enforcement.

1. Amendment Summary and Supersession Map

Version 1.0 already promised to preserve valid evidence after narrow repairs, distinguish product/test/environment/infrastructure failures, and use impact planning rather than repeated monolithic gates. Version 1.1 converts those principles into explicit normative mechanics.

v1.0 area	v1.1 amendment
1.2 Verification Promise	Adds mandatory semantic invalidation and carry-forward; preservation is no longer advisory language.
4 Core Domain	Adds evidence-disposition and closure-confidence vocabulary.
5 Design Principles	Adds smallest correct revalidation boundary and proof-preservation requirement.
14 Test Impact Analysis	Adds inverse impact analysis: what existing evidence remains valid after a change.
16 Status Language	Adds evidence-preserved/rebound statuses and environment-qualified evidence.
17 Failure Classification	Binds each failure class to the legal repair scope.
19 Evidence Architecture	Adds immutable carry-forward receipts and rebinding manifests.
20 Release Confidence	Permits confidence composition from preserved + fresh evidence.
25 Governance	Adds excessive-revalidation metrics and review triggers.

Hard boundary

Version 1.1 does not allow stale evidence. It requires Sounding Line to prove why retained evidence is still valid. “Do not rerun it because it is expensive” is invalid. “Do not rerun it because no producer, consumer, contract, environment, fixture, or resource identity on which that evidence depends changed” is valid.

2. New Canonical Vocabulary

Term	Definition
Evidence Invalidation Manifest	Machine-readable explanation of which prior receipts are invalidated by a new source/policy/environment state and why.
Evidence Carry-Forward Receipt	New-source receipt referencing immutable prior evidence and proving its dependencies remain unchanged.
PRESERVED	Prior receipt remains semantically valid for the new decision through a carry-forward proof.
INVALIDATED	Changed dependency means the prior receipt cannot contribute to the new decision.
REBOUND	Evidence meaning is unchanged but source-bound metadata or generated-record identity is updated through a validated rebinding step.
SUPERSEDED	A stronger/newer valid receipt replaces a prior receipt for decision purposes; the historical receipt remains immutable.
Material Product Change	Production behavior, authoritative contract, persistence, security/privacy, or runtime semantics changed.
Test-Only Change	Only test logic, locator, assertion plumbing, test timing budget, or test fixture orchestration changed; production behavior did not.
Evidence-Only Change	Only generated evidence, completion record, catalog status, or source-bound evidence metadata changed.
Environment-Qualified Evidence	Receipt whose validity includes an environment class such as reference CI, normal developer workstation, or resource-contended workstation.

3. Semantic Evidence Invalidation

Evidence invalidation is determined by dependency semantics, not by SHA inequality alone. A commit hash changes whenever any tracked file changes. Verification meaning changes only when a dependency of the receipt changes.

```
NEW SOURCE STATE
|
+--> changed files/contracts/policy/environment
|
+--> dependency graph analysis
|
+--> INVALIDATED receipts -> rerun
+--> PRESERVED receipts -> carry forward
+--> REBOUND records -> regenerate/rebind
+--> SUPERSEDED receipts -> retain historically, omit from decision
```

3.1 Required invalidation inputs

Changed production files and their direct/transitive consumers.

Changed verification contracts, suite definitions, fixture versions, and test ownership metadata.

Changed schemas, migrations, generated clients, lockfiles, toolchain identity, or resource contracts.

Changed security/privacy/persistence/progression authority or public projection behavior.

Changed environment identity when the receipt depends on browser, OS, provider, hardware, timing, or capacity behavior.

Changed release policy or gate requirements.

3.2 Explicit non-invalidators

Unrelated source changes with no path to the receipt's protected contracts.

Documentation wording that does not change policy or executable examples.

Generated index/catalog regeneration when the owning machine-readable product contract is unchanged.

A different project landing on main when impact analysis proves no shared contract, schema, test infrastructure, fixture, or runtime dependency changed.

4. Test-Only and Evidence-Only Invalidation Ceiling

A test-only change may not cause a full repository revalidation merely because the source SHA changed. The planner must cap invalidation at the changed test family and any direct evidence consumers unless it can explain a broader dependency.

Change	Default invalidation ceiling	Broaden only when
Locator/assertion synchronization in one E2E file	Owning browser suite + direct finalizer input	Shared helper, fixture, route contract, or semantic assertion library changed.
Timeout budget with diagnosed environment cause	Owning suite timing evidence + finalizer	Timeout encodes a product performance contract or shared runtime policy.
Test fixture repair	Suites using that fixture + dependent decision nodes	Fixture is shared across families or changes authoritative seeded state.
Feature Catalog/status record only	Catalog/docs/policy validation + finalizer	Record changes executable product or release-gate semantics.
Completion receipt wording only	Docs/evidence schema checks	Receipt format/reader behavior changed.

Prohibited over-invalidation

A test-only edit to ``tests/e2e/project-helm-phase1.spec.ts`` may not automatically invalidate unrelated database, Community, Sealed Hold, Drydock, or migration proof. A broader rerun requires a written dependency explanation produced by the planner.

5. Failure Classification Controls Legal Repair Scope

Failure class	Permitted repair scope	Forbidden response
PRODUCT_DEFECT	Smallest responsible production scope plus regression test and affected evidence.	Changing only the test to hide a product failure.
TEST_DEFECT	Owning test/assertion/fixture and directly affected evidence.	Changing production behavior without independent product-defect evidence.
ENVIRONMENT_DEFECT	Reference environment, environment declaration, provisioning, or rerun in qualified environment.	Changing product semantics to accommodate contaminated local timing.
INFRASTRUCTURE_DEFECT	Sounding Line runtime/broker/executor/infrastructure owner.	Feature project refactor unless product defect separately proven.
INFRASTRUCTURE_CONTENTION	Scheduler/broker queue; wait or route to another lane.	Editing test/product to “fix” an occupied port or lease.
TIMEOUT	Diagnose product hang, test wait, environment pressure, or budget mismatch first.	Blind timeout increase or retry loop.
STALE_TEST	Migrate assertion to current authoritative contract while preserving risk.	Restoring retired behavior just to make an old test pass.

The classification must be recorded before source mutation when the distinction changes which owner is allowed to repair the failure.

6. Evidence Carry-Forward and Rebinding

Prior receipts remain immutable and source-bound. Carry-forward never edits the old receipt. It creates a new source-bound proof that references the old receipt and the invalidation analysis.

```
Receipt R1 @ SHA A = PASSED/CLEAN
A -> B diff analysis = no dependency of R1 changed
CarryForward CF1 @ SHA B references R1
CF1 disposition = PRESERVED
Finalizer @ B consumes CF1 + fresh invalidated-node receipts
R1 remains immutable historical evidence
```

Carry-forward record field	Requirement
newSourceSha	Exact candidate or integrated SHA receiving the preserved evidence.
priorReceiptDigest	Digest of immutable source receipt.
protectedContracts	Contract IDs/suite IDs proven by prior receipt.
dependencyFingerprint	Source/policy/fixture/resource/environment dependencies checked.
disposition	PRESERVED, INVALIDATED, REBOUND, or SUPERSEDED.
reason	Machine-readable reason plus human-readable explanation.
plannerVersion	Exact policy/planner identity that authorized the disposition.
createdAt / actor	Auditable time and authority.

7. Release Confidence Composition

A release or mainline decision may be composed from fresh receipts and carried-forward receipts when every mandatory evidence obligation resolves to one valid source-bound producer.

Evidence state	May contribute to gate?	Condition
Fresh PASSED/CLEAN	Yes	Receipt matches exact source/policy/environment requirement.
PRESERVED via carry-forward	Yes	Invalidation manifest proves no dependency changed.
REBOUND	Yes	Only source-bound record/generation identity changed; semantic proof preserved.
SUPERSEDED	No direct contribution	Use the replacing receipt.
INVALIDATED	No	Must be rerun or replaced.
Environment-contaminated timing receipt	Only for non-timing semantics	Decision must not use it to prove performance/timing contract.

Confidence rule

A new SHA is not a reason to discard valid proof. A changed dependency is. Sounding Line must be able to explain the difference.

8. Environment Qualification and Timing Evidence

Evidence that depends on wall-clock timing, cold compilation, CPU/GPU contention, memory pressure, browser startup, or thermal state must identify its environment class. Functional semantic assertions may remain useful even when timing evidence is contaminated.

Environment class	Use
REFERENCE_CI	Preferred authority for hosted timing and clean-runner browser behavior.
DEVELOPER_NORMAL	Local proof when the workstation is within normal capacity profile.
DEVELOPER_CONTENDED	Functional diagnostics may be retained; timing/performance conclusions are not authoritative.
EXTERNAL_PROVIDER	Proof includes provider identity/region/version and provider availability state.

A game, render workload, thermal throttling, IDE indexing storm, or other heavy interactive workload is not a product defect. Sounding Line must classify the local environment and either defer timing-sensitive proof, route it to a reference runner, or narrow the evidence claim.

9. Closure Efficiency as Verification Quality

Verification depth remains non-negotiable, but unnecessary revalidation is itself a quality defect because it delays integration, increases contention, and raises the probability of mainline drift invalidating otherwise complete work.

Metric	Required interpretation
Full-gate count per phase	Track separately from focused reruns. Repeated full gates without product changes require review.
Evidence preservation ratio	Percentage of mandatory obligations satisfied by valid carry-forward after narrow change/main advance.
Closure latency	Time from locally complete to accepted mainline; separate queue time, execution time, rework time, and mainline-drift time.
Over-invalidation ratio	Evidence rerun despite no dependency change. Target should trend toward zero.
Queue wait	Part II scheduling metric; not product implementation time.

10. Part I v1.1 Acceptance Additions

Planner can produce a deterministic Evidence Invalidation Manifest.

Carry-forward receipts are immutable, source-bound, and auditable.

Test-only change does not automatically invalidate unrelated suites.

Failure class changes the legal repair owner/scope.

Environment-contaminated timing evidence cannot be mistaken for product failure.

Finalizer can compose fresh and preserved evidence without weakening mandatory proof.

Historical receipt remains immutable; preserved evidence is represented by a new derived record.

Policy tests prove broad rerun is rejected when no dependency explanation exists.

Final Part I v1.1 rule

Preserve proof when its meaning is unchanged. Rerun proof when its dependencies changed. Never use SHA churn, test-harness churn, or environmental slowness as a substitute for semantic impact analysis.

R. References and Amendment Closure

Base authority: Project Sounding Line Part I - Software Verification Architecture, Version 1.0, July 28, 2026. Companion authority: Parts II and III Version 1.1 Amendment Editions. Operational motivation includes Project Helm Phase 1 closure evidence from August 2026, treated as incident history rather than product semantics.

All unaffected Version 1.0 clauses remain governing. Where Version 1.0 and Version 1.1 differ on evidence invalidation, carry-forward, or failure-class repair scope, Version 1.1 controls.